Федерация спортивного программирования России · спецтрек

FSP Talent

Цифровая платформа подбора ИТ-специалистов с обратной механикой и верифицированным профилем достижений участника ФСП

Сопроводительная документация · 05.10.2026

Содержание

1. Обзор решения
2. Архитектура
3. Механика подбора
4. Механика тестирования
5. Интеграция с реестром ФСП
6. API
7. Сборка и развёртывание
8. Интерфейс

Документ собран из исходных файлов репозитория (`README.md` , `docs/*.md`). Отчёты валидации — `evaluation/results/` , API — Swagger UI по адресу `/api/docs` .

1. Обзор решения

Цифровая платформа с **обратной механикой подбора**: кандидат проходит тест на заявленный грейд и получает объективную категорию («Backend · Middle»), профиль автоматически обогащается достижениями Федерации спортивного программирования, а **работодатель сам** описывает потребность, получает ранжированную и объяснимую подборку и направляет конкретному человеку приглашение с зарплатной вилкой.

> MVP для спецтрека ФСП «Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника». Всё ниже работает «из коробки» на демо-данных.

Русский и английский. Интерфейс, ответы сервера (ошибки, объяснения подбора, рекомендации по тесту, справочники, письма) и весь банк заданий доступны на двух языках. Язык определяется по браузеру, переключается RU / EN на любом экране и сохраняется в учётной записи. Тест можно проходить на любом языке и переключать по ходу — варианты ответов и калибровка общие (docs/architecture.md).

Быстрый старт

```
docker compose up --build
```

Приложение	**http://localhost:8080**
API (Swagger)	http://localhost:8000/api/docs
Почта (коды подтверждения)	http://localhost:8025

Без Docker — см. docs/deployment.md (Python 3.12 + Node 20, SQLite по умолчанию).

Демо-аккаунты (пароль у всех: `demo12345`)

Роль	E-mail	Что показывает
Кандидат	`candidate@demo.ru`	Backend · Middle, 3 достижения ФСП, входящие приглашения
Кандидат	`fresh@demo.ru`	Профиль заполнен, тест не пройден — можно сразу пройти тестирование
Работодатель	`hr@demo.ru`	«Технологии Будущего»: 3 потребности, подборки, приглашения
Работодатели	`jobs@severcode.demo`, `team@datalab.demo`, `hr@oblakotech.demo`, `people@mediaplatform.demo`	Другие компании
Кандидаты	`cand01@talent.demo` … `cand48@talent.demo`	48 синтетических кандидатов

На главной странице есть кнопки входа в демо-аккаунты. Для живой регистрации с ФСП: зарегистрируйтесь с e-mail `olymp@demo.ru` (достижения подтянутся автоматически) или привяжите свободный ФСП ID `FSP-200001` / `FSP-200002` / `FSP-200003` (код подтверждения показывается в демо-режиме).

Сквозной сценарий демонстрации

1. **Кандидат**: «Регистрация» → код подтверждения (в демо виден на экране и в Mailpit) → профиль (можно загрузить PDF-резюме) → «Тестирование»: опрос — Backend, Middle → «Начать тест» → 20 вопросов → результат: грейд, категория, шкальный балл, компетенции, уверенность → «Достижения ФСП» → «Настройки»: включить публикацию профиля.
2. **Работодатель** `hr@demo.ru`: «Новая потребность» → описать текстом («Middle Python, FastAPI, PostgreSQL, Docker, 180–250 тыс., удалёнка») → «Разобрать описание» → «Получить подборку» → рекомендованные категории и рейтинг с объяснением → «Профиль и объяснение» (вклад факторов, контакты скрыты) → «Пригласить» (вилка обязательна).
3. **Кандидат**: «Приглашения» → компания, предложение, **зарплата до ответа**, «почему выбрали вас» → «Принять» → у работодателя открываются ФИО, e-mail, телефон, Telegram.

Дополнительно: сравнение кандидатов «почему A выше B», банк кандидатов с фильтрами, публичные вакансии и отклики, короткие задания от работодателей, жалоба на приглашение, PDF-профиль.

Видео сценария (≈5 мин, с субтитрами о механиках тестирования и подбора) записывается на запущенном стенде: `pip install playwright imageio-ffmpeg && python -m playwright install ffmpeg`, затем `python scripts/record_demo.py` → `presentation/FSP_Talent_demo.mp4` (демо-данные сбрасываются до и после записи).

Ключевые механики

Подбор (docs/matching.md). Объяснимая формула из 9 компонентов: стек через граф навыков с учётом *доказательности* (подтверждён ли навык тестом), подтверждённый грейд, результат теста относительно требуемого уровня, специализация, ФСП (нет истории = нейтральная база, без штрафа), актуальность, зарплата, опыт, формат + мягкое «И» для обязательных навыков. Каждый процент разложен на вклады факторов; есть сравнение A vs B; подборка сохраняется и уточняется фильтрами.

Тестирование (docs/testing.md). Банк из 252 калиброванных заданий и 25 параметрических семейств (новые числа для каждого кандидата) по 4 специализациям и 21 компетенции. Индивидуальный вариант собирается по единому плану (blueprint) — одинаковые квоты компетенций и профиль сложности; оценка по IRT (3PL, EAP) делает результаты разных вариантов сопоставимыми. Детектор утечки (person-fit + разрыв «статические vs параметрические задания») требует подтверждающей формы. Кулдауны: повтор через 30 дней, смена грейда — раз в 90 дней, тест ниже — сразу после провала, тест выше — сразу после уверенного результата.

ФСП (docs/fsp-integration.md). Порт `FSPRegistry` с мок-адаптером и HTTP-адаптером, мок-API реестра, автопривязка по подтверждённому e-mail, ручная привязка ФСП ID с подтверждением владения, совместимость с Keycloak (OIDC-claims в JWT).

Валидация (воспроизводимо, без БД)

```
python evaluation/evaluate_tests.py      # механика тестирования
python evaluation/evaluate_matching.py   # бенчмарк подбора: P@K, R@K, MRR, NDCG
python evaluation/tune_matching.py       # подбор гиперпараметров с проверкой на отложенных данных
```

Тестирование (симуляция с известной истиной, 1 600 кандидатов)	
Точность грейда / взвешенная κ / в пределах ±1	80,5% / 0,898 / 99,9%
Тест–ретест, корреляция θ на двух независимых вариантах	0,866

Тестирование (симуляция с известной истиной, 1 600 кандидатов)	
Эквивалентность вариантов (SD ожид. балла): blueprint vs случайный отбор	0,013 vs 0,045
Junior выучил ответы 10 «слитых» вариантов Middle → доля сдавших: единый тест / наш подход	98% / **1,3%**
Ложные срабатывания детектора утечки	4,4%

Подбор (60 потребностей × 600 кандидатов)	P@5	NDCG@10	MRR
Поиск по ключевым словам резюме	0.377	0.461	0.655
Ключевые слова + заявленный грейд	0.700	0.692	0.817
FSP Talent	**0.880**	**0.808**	**0.933**

Отчёты: `evaluation/results/*.md`.

Архитектура

```
frontend/   React 19 + TypeScript + Vite; типы API генерируются из OpenAPI
  src/i18n/           словари RU / EN, плюрализация, форматирование, переключатель языка
  src/**/*.test.ts*   107 тестов (Vitest + Testing Library)
backend/    FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL (SQLite локально)
  app/api/            REST-роутеры (без бизнес-логики)
  app/services/       сценарии: auth, кандидат, работодатель, приглашения, приватность, задания
    testing/          Testing Engine: банк → blueprint → генерация → IRT → правила грейдов
    matching/         Matching Engine: признаки → скоринг → объяснение → снимок подборки
    fsp/              порт реестра ФСП, адаптеры, сила сигнала
    nlp/              разбор PDF-резюме и текстового описания потребности (RU / EN)
  app/i18n/           язык запроса (Accept-Language), каталоги сообщений RU / EN
  app/repositories/   доступ к данным
  app/models/         23 таблицы, миграции в alembic/
  app/domain/         справочники и банк заданий
  app/seed/           демо-данные (кандидаты проходят настоящий движок тестирования)
  tests/              381 тест (pytest): движки, политика грейдов, приватность, локализация, сквозной
сценарий
evaluation/  офлайн-валидация тестирования и подбора
docs/        архитектура, механики, интеграция ФСП, API, развёртывание
```

Подробнее: docs/architecture.md · API: docs/api.md · развёртывание: docs/deployment.md. Вся документация одним файлом — docs/FSP_Talent_documentation.pdf (собирается `scripts/build_docs_pdf.py`).

Проверки

```
cd backend && pytest -q && ruff check app tests scripts
cd frontend && npm test && npm run typecheck && npm run lint && npm run build
python scripts/smoke_test.py --base http://localhost:8080
```

Те же проверки (плюс применение и откат миграций и короткий бенчмарк подбора) запускаются в CI на каждый push и pull request: .github/workflows/ci.yml.

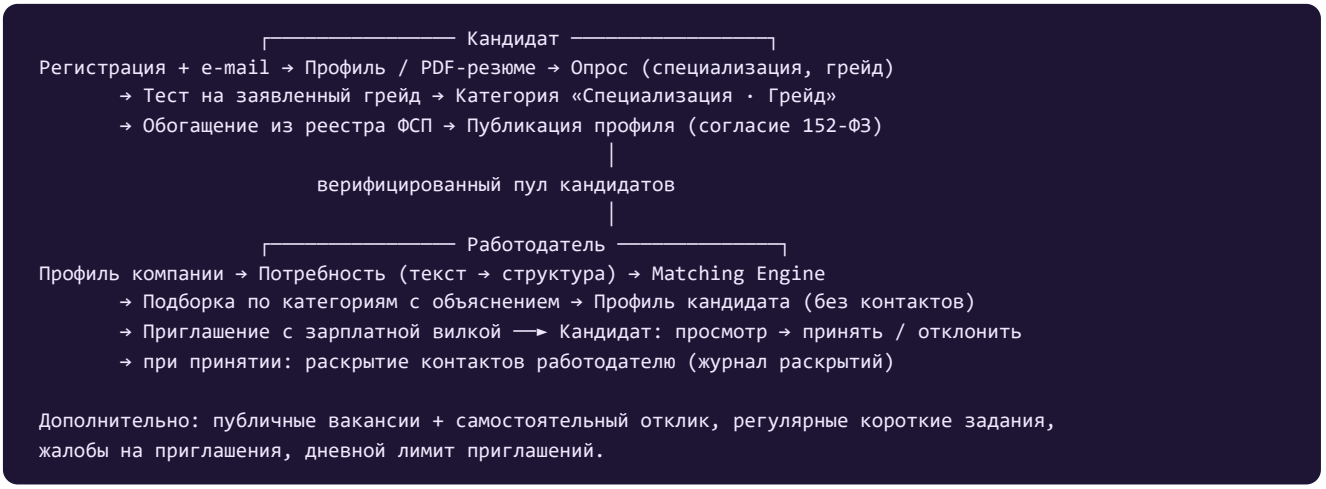
Ограничения MVP

- Реестр ФСП — синтетический мок (реальный API не предоставлен); интеграция заменяется адаптером.

- Параметры заданий — экспертная калибровка; процедура перекалибровки по ответам реализована и проверена на симуляции, но требует пилота на реальных кандидатах.
- Веса матчинга подобраны на синтетическом бенчмарке; после запуска — дообучение на исходах приглашений.
- Демо-режим показывает код подтверждения в интерфейсе (`EMAIL_DEV_EXPOSE_CODE=false` отключает).
- Пользовательские тексты (резюме, описания вакансий, названия соревнований ФСП) не переводятся. Английская версия заданий использует ту же калибровку — на пилоте стоит проверить эквивалентность заданий между языками (DIF-анализ).
- Защита от недобросовестных работодателей и интеграция с ATS — концепция + базовые меры (лимиты, валидация вилок, жалобы), см. docs/architecture.md.

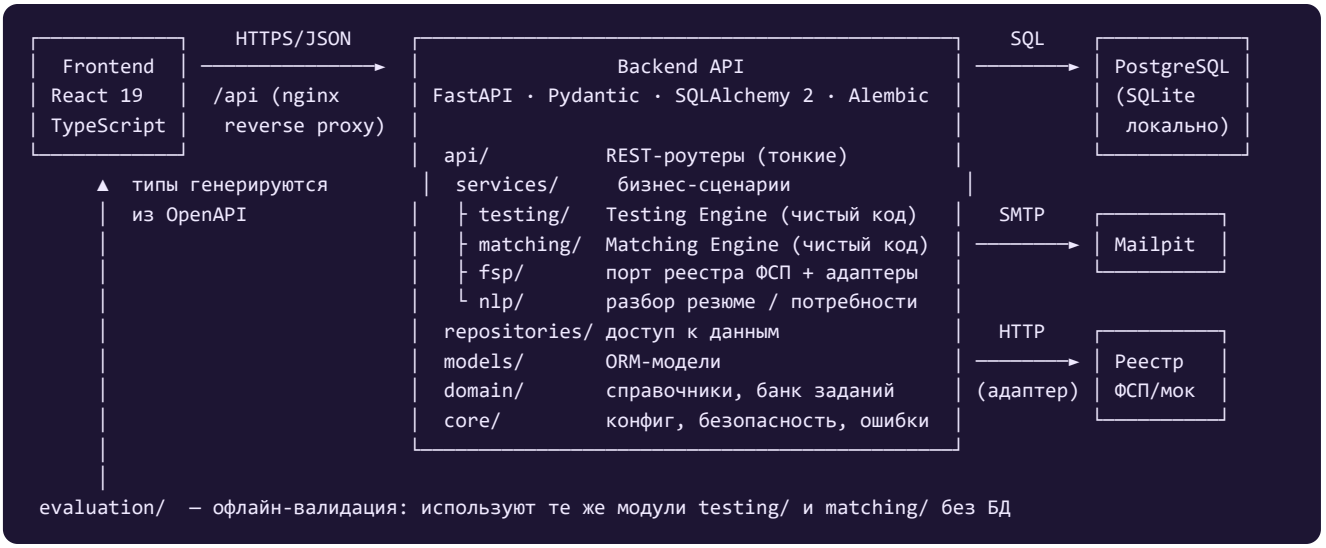
2. Архитектура

1. Функциональная архитектура



Инициатива контакта — у работодателя (модель «обратного подбора», как в LHH): кандидат не рассылает отклики, а получает предложения с условиями.

2. Компонентная архитектура



Монолит с явными слоями и слабо связанными модулями (допускается ТЗ). Testing Engine и Matching Engine — **чистые функции над dataclass-ами** (`ItemSpec` , `CandidateFeatures` , `JobFeatures`), не зависят от БД и HTTP: тот же код работает в API, в seed-данных и в скриптах валидации. Роутеры не содержат бизнес-логики — только валидацию входа (Pydantic) и вызов сервиса.

Слои backend

Слой	Каталог	Ответственность
API	`backend/app/api`	REST-эндпоинты, зависимости аутентификации и ролей, документация ошибок
Схемы	`backend/app/schemas`	Pydantic-контракты запросов/ответов → OpenAPI → TypeScript-типы фронтенда

Слой	Каталог	Ответственность
Сервисы	`backend/app/services`	Сценарии: регистрация, тестирование, подбор, приглашения, приватность
Движки	`services/testing`, `services/matching`	Генерация форм, IRT-оценка, правила грейдов; скоринг, объяснения
Репозитории	`backend/app/repositories`	Запросы с жадной загрузкой для ранжирования целых категорий
Модели	`backend/app/models`	23 таблицы, ограничения целостности (CHECK, UNIQUE, FK)
Домен	`backend/app/domain`	Справочники специализаций/грейдов/навыков, банк заданий

3. Модель данных

```
User 1—1 CandidateProfile 1—* CandidateSkill *—1 Skill
                │ 1—1 FSPProfile 1—* FSPAchievement
                │ 1—* TestAttempt 1—* TestItem 1—1 TestAnswer        (TestItem → Question)
                │                   1—1 TestResult
User 1—1 EmployerProfile 1—* Job (потребность / вакансия) 1—* JobSkill *—1 Skill
                                  │ 1—* MatchResult *—1 CandidateProfile   (снимок ранжирования)
EmployerProfile 1—* Invitation *—1 CandidateProfile   (Invitation → Job, необязательно)
Job 1—* Application *—1 CandidateProfile
EmployerProfile 1—* ShortTask 1—* TaskSubmission *—1 CandidateProfile
ContactAccessLog (employer, candidate, reason) — журнал раскрытий контактов
Specialization 1—* Competency (план теста) · Grade (порог θ) · Question (версионируется)
```

Ключевые решения: * **Job** объединяет «потребность» (приватную, для подбора) и «вакансию» (`is_public`) — приглашать можно без публикации, как требует ТЗ. * **MatchResult** хранит снимок ранжирования: фильтры уточняют подборку, не теряя её и отметки «избранное»; снимок пересчитывается при изменении потребности или по TTL (5 минут). * **TestItem** хранит отрисованный вариант вопроса (перемешанные варианты, параметры семейства) и правильный ответ — он никогда не уходит клиенту. Вопросы банка версионируются (`code` + `version`). * Категория кандидата хранится в `verified_specialization_id` + `verified_grade_id` и меняется **только** сервисом тестирования. * Зарплата обязательна: `CHECK salary_from > 0 AND salary_to >= salary_from` в `jobs` и `invitations` .

Миграции — Alembic (`backend/alembic/versions`), одинаково работают на PostgreSQL и SQLite.

4. Безопасность и приватность

- Пароли — bcrypt; токены — JWT HS256 с OIDC-совместимыми claims (`sub` , `email` , `iss` , `realm_access.roles`). Переход на **Keycloak / ФСП ID** — замена проверки подписи на JWKS (RS256) в `core/security.py::decode_access_token` ; поле `users.external_subject` готово для связи учёток.
- RBAC: зависимости `Candidate` / `Employer` в `api/deps.py` ; владелец ресурса проверяется в сервисах (чужая попытка теста, чужая потребность → 404, чтобы не раскрывать существование).
- Регистрация с подтверждением e-mail (6-значный код, хеш в БД, TTL 30 мин); вход без подтверждения запрещён. В демо-режиме (`EMAIL_DEV_EXPOSE_CODE=true`) код также возвращается API.
- 152-ФЗ: согласие на обработку — обязательно при регистрации; согласие на публикацию — отдельно, без него профиль не участвует в поиске; настройки видимости (фамилия, ФСП, зарплатные ожидания).
- Контакты кандидата раскрываются работодателю **только** после принятия его приглашения или самостоятельного отклика (`services/privacy.py`), каждое раскрытие пишется в `contact_access_log` . До этого работодатель видит «Имя Ф.».

- Правильные ответы теста не покидают сервер; ответы валидируются по типу задания.
- Защита от спама: дневной лимит приглашений на работодателя (429), запрет дубликатов активных приглашений (409), ограничение ширины зарплатной вилки (≤ ×3), жалоба кандидата на приглашение.

5. Масштабирование и интеграции

- Stateless API (несколько воркеров uvicorn), состояние — в PostgreSQL. Ранжирование категории — O(N) по верифицированным кандидатам смежных специализаций с жадной загрузкой навыков/ФСП; снимок кэшируется в `match_results`. Для десятков тысяч кандидатов — предварительный отбор по категории в SQL и фоновой пересчёт снимков.
- Интеграция с реестром ФСП — через порт `FSPRegistry` (см. fsp-integration.md).
- **ATS (концепция).** Исходящие вебхуки на события `invitation.accepted` и `application.created` с профилем кандидата в стандартизированном JSON (тот же формат, что PDF-профиль); входящий API-ключ компании для выгрузки подборки по потребности (`GET /employer/requirements/{id}/matches`).
- **Защита от фиктивных вакансий (концепция).** Скоринг доверия работодателя: верификация домена e-mail/ИНН, доля принятых приглашений, жалобы кандидатов (`invitations.reported`), аномалии вилок относительно рынка категории; при низком доверии — модерация приглашений и снижение лимита. В MVP реализованы: лимиты, валидация вилок, жалобы.

6. Используемые библиотеки (версии)

Backend (Python 3.12): fastapi 0.142.2, uvicorn 0.54.0, sqlalchemy 2.1.3, alembic 1.20.0, pydantic 2.13.5, pydantic-settings 2.15.0, psycopg 3.3.6, pyjwt 2.15.1, bcrypt 5.0.0, email-validator 2.3.0, python-multipart 0.0.32, pypdf 6.19.0, httpx 0.28.1; dev: pytest 9.1.1, ruff 0.16.10.

Frontend (Node 22): react 19.3.0, react-dom 19.3.0, react-router-dom 7.18.4, lucide-react 1.52.0, vite 8.3.2, typescript 5.9.3, @vitejs/plugin-react 6.1.1, openapi-typescript 7.13.0, eslint 10.12.0, typescript-eslint 8.71.0; тесты: vitest 5.0.3, @vitest/coverage-v8 5.0.3, jsdom 30.1.2, @testing-library/react 16.3.3, @testing-library/dom 10.4.2, @testing-library/jest-dom 7.0.1, @testing-library/user-event 14.6.7.

Инфраструктура: PostgreSQL 16 (alpine), nginx 1.27 (alpine), Mailpit 1.21, Docker Compose. Все компоненты — open source (MIT / Apache-2.0 / BSD / PostgreSQL License).

7. Локализация (RU / EN)

Выбор языка. Порядок: явный выбор на устройстве (переключатель RU / EN, `localStorage`) → язык учётной записи (`users.locale` , восстанавливается при входе с другого устройства) → язык браузера (`navigator.languages`) → русский. Выбор, сделанный вошедшим пользователем, сохраняется в учётной записи (`PUT /api/auth/me/locale`); при регистрации записывается язык, на котором она прошла.

Фронтенд (`frontend/src/i18n`). Словари по разделам (`locales/ru/*` , `locales/en/*`); английский словарь типизирован от русского, поэтому пропущенный ключ — ошибка компиляции. Плюрализация — `Intl.PluralRules` (ru: one / few / many, en: one / other), числа, деньги и даты — `Intl` (`ru-RU` , `en-GB` : порядок «день-месяц-год» в обоих языках). API-клиент отправляет `Accept-Language` с каждым запросом, а `useApi` перезапрашивает данные при смене языка — тексты, сформированные сервером (объяснения, причины, справочники), переключаются вместе с интерфейсом. `<html lang>` обновляется для скринридеров. Тест `coverage.test.ts` запрещает русский текст в компонентах вне словарей.

Бэкенд (`backend/app/i18n`). ASGI-middleware определяет язык запроса по `Accept-Language` (с учётом q-весов) и кладёт его в контекстную переменную; сервисы вызывают `t(key, **params)` . Ответ помечается `Content-Language` и `Vary: Accept-Language` . Ошибки 422 от Pydantic переводятся с названием поля.

Хранимые тексты не хранятся готовыми. Снимок подборки (`match_results`) и причины в приглашении сохраняются как объекты сообщений `{code, params}` с языконезависимыми параметрами (id, числа, имена собственные; деньги — `{"money": n}` , навык — `{"skill": slug}` , дата — `{"date": …}`) и отрисовываются при чтении: работодатель на английском и кандидат на русском видят одну и ту же подборку каждый на своём языке. Рекомендация к результату теста выводится из сохранённого исхода. Старые записи с готовым текстом отображаются как есть.

Банк заданий. Все 252 статических задания и 25 параметрических семейств есть на английском. Перевод хранится в `questions.i18n` с тем же порядком вариантов, поэтому ключ ответа и IRT-калибровка общие; семейства генерируют обе версии из одних случайных параметров. В попытке (`test_items.i18n`) хранятся обе версии с одинаковой перестановкой вариантов — язык можно сменить посреди теста, ответы остаются верными. Тесты проверяют полноту перевода, совпадение числа вариантов, неизменность фрагментов кода и одинаковые числа в обеих версиях параметрических заданий.

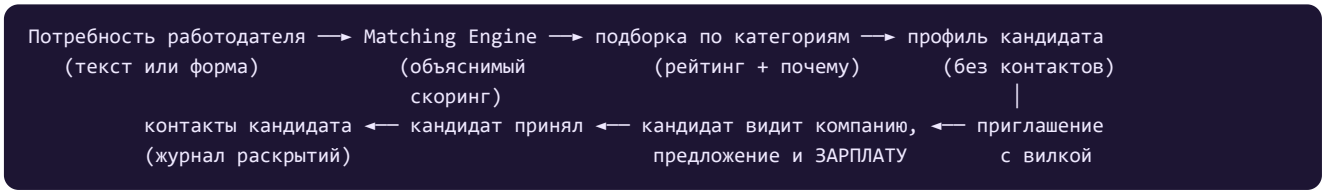
Разбор текста. NLP-разбор потребности и резюме понимает оба языка: «желательно Redis» и «Redis is a plus», «190–260 тыс.» и «from 150 to 200k», «опыт от 2 лет» и «2+ years».

Пользовательские данные (резюме, описания вакансий, названия соревнований из реестра ФСП) показываются так, как их ввёл автор.

3. Механика подбора

1. Сценарий: обратный подбор

Кандидат **не откликается** на вакансии. Инициатива — у работодателя:



- Потребность не обязана быть опубликованной вакансией: приглашать можно напрямую.
- Описание потребности можно ввести текстом — NLP-разбор извлекает специализацию, грейд, стек, вилку, формат и опыт (`services/nlp/extractor.py`); работодатель проверяет и уточняет поля. Навыки после «желательно / приветствуется» или перед «будет плюсом» попадают в nice-to-have, остальные — в must-have; вилка «190–260 тыс.» понимается целиком, телефон не принимается за зарплату.
- Подборка сохраняется (`match_results`), фильтры уточняют её **без потери** (грейд, категория, обязательный навык, наличие ФСП, минимальный %, избранное); пересчёт — при изменении потребности, по TTL или кнопкой.
- Работодатель видит «рекомендованные категории» (Backend · Middle — 10 кандидатов, средний и максимальный %), затем конкретных людей в категориях.
- Сравнение «почему A выше B» — разложение разницы по факторам (`GET …/compare`).
- Приглашение: должность, описание, **обязательная вилка от–до в ₽**, формат, способ связи; статусы `sent → viewed → accepted | rejected`, а также `expired` (14 дней) и `withdrawn`.
- Кандидат видит условия и «почему выбрали вас» до ответа; контакты раскрываются только после принятия (или если кандидат сам откликнулся на вакансию).

Дополнительно: банк кандидатов (вся база с фильтрами, сортировка внутри категории по силе подтверждённого профиля), публичные вакансии с обратным матчингом для кандидата (% совпадения у каждой вакансии), отклики, короткие задания.

2. Категоризация

Категория = **подтверждённая специализация · подтверждённый грейд** (Backend · Middle и т.д.), присваивается только тестированием (см. testing.md). Кандидат без подтверждённой категории или без согласия на публикацию в подборки не попадает. Внутри категории кандидаты ранжируются по силе подтверждённого профиля:

```
profile_strength = 0.6 · σ(1.7·(θ − порог грейда)) + 0.25 · ФСП + 0.15 · актуальность
```

3. Формула совпадения

```
match = gate_must × Σ_k w_k · s_k / Σ_k w_k        по ДОСТУПНЫМ компонентам k,  s_k ∈ [0, 1]
```

k	Компонент	Вес	Как считается
skills	Стек и навыки	0.26	0.75·покрытие must + 0.25·nice. Каждый требуемый навык ищется через **граф навыков** (FastAPI ≈ Flask 0.7, PostgreSQL ≈ MySQL 0.65, React ≈ Next.js 0.75 …) и умножается на **доказательность**: заявленный уровень × (0.75 + 0.25·балл теста по компетенции, которая этот навык проверяет)
grade	Грейд	0.22	совпадает 1.0; выше на 1 — 0.6; ниже на 1 — 0.4 (0.6 если тест показал готовность к повышению); разница ≥ 2 — исключение
test	Результат теста	0.16	0.65·σ(1.7·(min(θ, порог+1) − порог требуемого грейда)) + 0.35·средний балл компетенций, релевантных стеку. Способность выше следующего грейда не даёт бонуса (переквалификацию учитывает «грейд»)
specialization	Специализация	0.10	совпадает 1.0; смежная — частично (Backend↔DevOps 0.35, Backend↔Data 0.25…); несмежная — исключение
fsp	Достижения ФСП	0.10	0.3 + 0.7·сила; **нет истории → 0.3 (нейтрально, без штрафа)**
freshness	Актуальность профиля	0.06	0.5^(дней с последней активности / 120); активность — тест, ответ на приглашение, короткое задание
salary	Зарплатные ожидания	0.05	в вилке — 1; выше — линейный спад до 0 при превышении на 40%; не указаны — компонент недоступен
experience	Опыт	0.03	min(1, опыт / требуемый); требование не задано — недоступен
format	Формат работы	0.02	матрица совместимости remote/office/hybrid + город

- **Нет данных ≠ 0.** Недоступные компоненты (не указаны ожидания по зарплате, нет требования к опыту) исключаются, а веса перенормируются. ФСП всегда «доступен» через нейтральную базу.
- **Мягкое «И» для обязательных навыков**: `gate_must = min(1, (0.8 + 0.2·покрытие_must) / 0.92)` — отсутствие обязательных навыков масштабирует весь балл, чтобы сильный тест не маскировал отсутствие того, что работодатель явно требует.
- Жёсткие фильтры: несмежная специализация, разница грейдов ≥ 2, итог ниже 35%.
- При равенстве баллов выше тот, у кого выше θ, затем — сильнее ФСП.

Профили весов на выбор работодателя: **Сбалансированный** (по умолчанию), **Сначала стек** (skills 0.40), **Подтверждённый талант** (test 0.30, ФСП 0.18). Веса переопределяются через `MATCHING_WEIGHTS_JSON`. Код: `backend/app/services/matching/engine.py`, `config.py`.

Почему такие веса

- Стек (0.26) — главный язык, которым работодатель описывает потребность, но навык учитывается по доказательности, а не по факту упоминания в резюме.
- Грейд (0.22) — центральная часть категории; подобран на бенчмарке (п. 6): недо- и переквалификация — основные причины нерелевантной выдачи.
- Тест (0.16) — сопоставимая (IRT) мера уровня относительно требуемого грейда.
- Специализация (0.10) — в основном работает как фильтр, частичный балл только для смежных профилей.
- ФСП (0.10) — проверяемый сигнал практических навыков; заметно поднимает внутри категории, но не может перевесить стек и грейд; нейтральная база исключает дискриминацию тех, кто не участвовал.
- Актуальность, зарплата, опыт, формат — небольшие поправки (самоописанный опыт — сознательно 0.03).

4. Объяснимость

Каждый результат содержит: * **причины** — человекочитаемые утверждения с типом (✓ плюс / • нейтрально / ✗ минус): «Подтверждённый грейд Middle — как требуется», «Совместимость стека 82%: 4 из 4 обязательных навыков, 4 подтверждены тестом», «Близкие навыки: Flask вместо FastAPI», «Нет истории ФСП — сигнал не учитывается, без штрафа», «Ожидания 410 000 ₽ — выше вилки на 64%»; * **разложение балла** — оценка, вес и вклад каждого фактора в итоговые проценты (сумма вкладов = итог), множитель покрытия обязательных навыков; * **сопоставление навыков** — точное совпадение / близкий навык / отсутствует, подтверждён ли тестом; * **сравнение двух кандидатов** — разница вкладов по факторам и текстовое резюме («Разница +8 п. Преимущество: грейд (+8.8), стек и навыки (+2.3). Уступает: достижения ФСП (−1.2)»).

Кандидат в приглашении видит процент совпадения и положительные причины — почему выбрали его.

5. Приватность в подборе

До контакта работодатель видит «Имя Ф.», категорию, тест, навыки, ФСП, город, ожидания (если кандидат разрешил). Контакты — только после принятия приглашения или отклика. Скрытые кандидатом данные (ФСП, зарплата) не используются и в ранжировании.

6. Валидация (`python evaluation/evaluate_matching.py`)

Готового датасета нет, поэтому построен **синтетический бенчмарк со скрытой истиной**: 60 потребностей × 600 кандидатов. У кандидата есть скрытый «истинный» профиль (специализация, способность θ, реальные навыки); движок видит только наблюдаемое:

- резюме: истинные навыки с пропусками (10%) и **«приписанными» модными навыками своей области** у 40% кандидатов (keyword stuffing — именно проблема, описанная в ТЗ);
- самооценка грейда: завышение на уровень у 30%, занижение у 10%;
- результат теста: θ с погрешностью 20-заданной формы (SE ≈ 0.33) и баллы компетенций;
- ФСП: у ~50%, сильнее у сильных; зарплатные ожидания, формат, активность.

Метки релевантности 0–3 назначаются **экспертными правилами по скрытому профилю** (полезность найма: специализация × соответствие истинного грейда × реальное покрытие must-have × сила внутри грейда × совместимость по зарплате) — без использования весов движка. Релевантен кандидат с меткой ≥ 2.

Ранжировщик	P@5	P@10	R@10	MRR	NDCG@10
Случайный (та же специализация)	0.193	0.202	0.062	0.311	0.168
Поиск по ключевым словам резюме («классическая площадка»)	0.377	0.412	0.169	0.655	0.461
Ключевые слова + заявленный грейд	0.700	0.608	0.273	0.817	0.692
FSP Talent (сбалансированный)	**0.880**	**0.807**	**0.346**	**0.933**	**0.808**
— без тестирования (самооценка грейда)	0.813	0.747	0.325	0.929	0.771
— только подтверждённый грейд (без θ и компетенций)	0.887	0.817	0.352	0.992	0.833
— без графа навыков	0.873	0.782	0.343	0.958	0.824
— без сигнала ФСП	0.890	0.818	0.352	0.942	0.819
— без мягкого «И» для must-have	0.883	0.808	0.345	0.933	0.804

Ранжировщик	P@5	P@10	R@10	MRR	NDCG@10
Профиль «Сначала стек»	0.717	0.662	0.302	0.795	0.674
Профиль «Подтверждённый талант»	0.673	0.625	0.287	0.816	0.631

Выводы: * Движок существенно превосходит поиск по ключевым словам (NDCG@10 0.81 против 0.46) и даже поиск с учётом самооценки грейда (0.69): в топ-5 релевантны 88% кандидатов против 38% и 70%. * Тестирование даёт прирост: с самооценкой грейда вместо подтверждённого NDCG падает до 0.77. * Честно о слабых местах: в этой разметке сила внутри грейда влияет слабо (±15%), поэтому упорядочивание по θ внутри категории даёт −0.025 NDCG относительно «только грейд»; граф навыков и ФСП в синтетике почти нейтральны (метки не содержат ФСП, «близкие» навыки редки). Эти компоненты оставлены ради объяснимости и требований ТЗ («выше те, у кого сильнее подтверждённый профиль и есть опыт ФСП»), их веса умеренные. * **Справедливость к кандидатам без ФСП**: идеальные (метка 3) кандидаты без истории ФСП в среднем на 4,5-м перцентиле выдачи, с ФСП — на 3,6-м: ФСП помогает, но отсутствие не «топит».

Подбор гиперпараметров (`python evaluation/tune_matching.py`)

Сетка (минимум мягкого «И», сила тестовой доказательности навыков, доля компетенций в компоненте теста, 3 варианта весов) на обучающем мире (seed 11), проверка на **независимом** мире (seed 23):

Конфигурация	NDCG@10 train	NDCG@10 held-out	P@10 held-out	MRR held-out
Исходная экспертная (v1.0)	0.624	0.594	0.573	0.797
Лучшая на сетке (gate 0.8, evidence 0, grade 0.22)	0.832	0.809	0.792	0.967
Принятая в продукт (v1.2, evidence 0.25)	**0.808**	**0.786**	**0.770**	**0.956**

Прирост на независимых данных сохраняется (0.594 → 0.786), значит подбор не переобучен на одну синтетическую выборку. Сила тестовой доказательности навыков оставлена 0.25 вместо оптимальных 0: «навыки, подтверждённые тестом, весят больше» — продуктовый стимул проходить тест; цена — 0.023 NDCG на отложенных данных. Полный отчёт: `evaluation/results/tuning_report.md`.

Свой набор пар «вакансия — кандидат» можно проверить командой `python evaluation/evaluate_matching.py --dataset my_pairs.json` (формат — как у `evaluation/data/matching_benchmark.json`, экспортируемого скриптом).

7. Ограничения и развитие

- Веса подобраны на синтетике; после запуска — дообучение на реальных исходах (приглашение → принятие → найм) как learning-to-rank с теми же признаками; объяснимость сохраняется (линейная модель над интерпретируемыми компонентами).
- Граф навыков — экспертный; может пополняться эмбеддингами совместной встречаемости навыков.
- Регулярные короткие задания сейчас влияют на актуальность; оценки работодателей можно сделать отдельным компонентом «подтверждённые практические задачи».

4. Механика тестирования

Требования ТЗ и как они закрыты

Требование	Решение
Единый набор заданий не подходит — разойдётся	Каждому кандидату генерируется свой вариант из банка (сейчас 252 статических задания + 25 параметрических семейств)
Генерация нейросетью не гарантирует сопоставимую сложность	Варианты собираются по **единому плану (blueprint)**: одинаковое число заданий по компетенциям и одинаковый профиль сложности; оценка по **IRT** учитывает сложность конкретных заданий
Категория — не из самоописания, а через опрос → выбор грейда → тест	Категория `Специализация · Грейд` меняется только сервисом тестирования
Грейд не понижается принудительно; можно тест ниже/выше	Провал заявленного грейда → тест ниже доступен сразу; уверенный результат → тест выше без ожидания
Смена грейда ограничена по времени	Не чаще раза в 90 дней (`grade_locked_until`), повтор того же теста — через 30 дней

1. Банк заданий

`backend/app/domain/question_bank/` — 4 специализации × 5–6 компетенций, уровни сложности 1–5:

Специализация	Компетенции (доля в плане теста)
Backend	язык и рантайм 22%, БД и SQL 20%, API/HTTP/безопасность 18%, архитектура 18%, алгоритмы 12%, эксплуатация 10%
Frontend	JS/TS 26%, React и состояние 24%, HTML/CSS/a11y 18%, браузер и производительность 18%, тесты и сборка 14%
Data / ML	классическое ML 26%, статистика 20%, данные (SQL, pandas) 20%, DL 18%, MLOps 16%
DevOps	контейнеры и оркестрация 24%, Linux и сети 22%, CI/CD и IaC 20%, мониторинг 18%, облака и безопасность 16%

Метаданные задания (`questions`): `code` , `version` , `specialization` , `competency` , `difficulty` , `qtype` (single / multi / numeric), варианты, правильный ответ, IRT-параметры `a` , `b` , `c` , счётчик показов `exposure_count` , `family` для параметрических шаблонов. Изменение содержимого создаёт новую версию (старая деактивируется, попытки ссылаются на точную версию).

Параметрические семейства (`families.py` , `families_extra.py`) — шаблоны с фиксированной когнитивной операцией и сложностью, у которых при каждом показе меняются параметры: длина `range()` , число сравнений бинарного поиска, асимптотика циклов, строки после `GROUP BY … HAVING` , `LEFT JOIN` , специфичность CSS-селектора, батчинг `setState` , semver-диапазоны, precision/recall/F1, теорема Байеса, размер выхода свёртки, CIDR, chmod, бюджет простоя SLO, ёмкость узла Kubernetes, формула HPA и др. Ответ одного кандидата бесполезен следующему: у него другие числа. Семейства покрывают все 21 компетенцию.

Языки. Каждое задание банка есть на русском и английском (`question_bank/en_*.py` , по коду задания). Варианты ответа идут в том же порядке, фрагменты кода не меняются, поэтому правильный ответ и параметры IRT общие. Параметрические семейства формируют обе версии из одного набора случайных

параметров. Кандидат проходит тест на языке интерфейса и может переключить язык посреди попытки — вопросы придут в переводе, данные ответы останутся засчитанными.

2. Генерация варианта (`services/testing/generator.py`)

1. **Blueprint** для (специализация, целевой грейд, длина = 20): * квоты компетенций — по долям плана (метод наибольших остатков, минимум 1); * профиль сложности сконцентрирован у порогов решения грейда (там нужна максимальная информативность теста):

Цель	ур.1	ур.2	ур.3	ур.4	ур.5
Junior	20%	40%	33%	7%	—
Middle	—	20%	40%	33%	7%
Senior	—	—	25%	45%	30%
* уровни распределяются по компетенциям детерминированно (Сент-Лагю), так что у каждой					
компетенции есть лёгкие и трудные слоты. **План одинаков для всех кандидатов.**					
2. **Заполнение слотов**: для каждого слота случайно выбирается задание той же компетенции и					
сложности; приоритет — редко показанным (вес `1/(1+показы)` , контроль экспозиции), параметрические					
семейства предпочитаются (вес 3, утечка им не страшна). Задания, уже виденные кандидатом, исключаются.					
Если ячейка пуста — берётся ближайший уровень в сторону, компенсирующую накопленный сдвиг					
сложности, поэтому средняя сложность формы остаётся равной плановой.					
3. **Отрисовка**: случайный порядок заданий, перемешанные варианты ответов, новые параметры семейств.					
Правильные ответы хранятся на сервере (`test_items.correct`) и никогда не отправляются клиенту.					

3. Оценка (`services/testing/irt.py` , `scoring.py`)

- Модель 3PL: `P(верно | θ) = c + (1 − c) / (1 + e^(−1.7·a·(θ − b)))` ; начальные параметры — из экспертного уровня сложности (`b ∈ {−2, −1, 0, 1, 2}`) и формата (`a` = 1.0 / 1.3 / 1.4, `c` = 0.8/k для выбора из k, 0.05 для множественного выбора, 0 для числового ответа).
- Способность θ оценивается методом **EAP** (апостериорное среднее на сетке $\theta \in [-4; 4]$, априорно $N(0; 1.25^2)$) — конечна даже при всех верных/неверных ответах; апостериорное СКО — погрешность.
- Пороги грейдов на шкале θ: Junior −1, Middle 0, Senior +1. Публичная шкала **0–100 = 50 + 25·θ**, пороги читаются как 25 / 50 / 75 баллов. Шкальный балл сопоставим между разными вариантами, в отличие от «% верных».
- **Решение**: грейд подтверждён, если `P(θ ≥ порог | ответы) ≥ 0.5` ; «уверенный результат» (открывает тест на грейд выше без ожидания), если `P(θ ≥ порог следующего грейда) ≥ 0.8` . Уверенность решения — соответствующая апостериорная вероятность.
- Профиль компетенций — доля баллов по каждой компетенции (с частичным зачётом множественного выбора).

- Категория = `верифицированная специализация · подтверждённый грейд`.

4. Детектор утечки (защита от «слитых» ответов)

Кандидат, выучивший ответы, отвечает неправдоподобно: верно на трудные «известные» задания и неверно на свежие. Два статистических признака (`scoring.py`):

- **person-fit lz*** (Drasgow, 1985) — стандартизованное правдоподобие паттерна ответов; порог −2;
- **разрыв способностей**: θ по статическим заданиям минус θ по параметрическим (неутекаемым), в единицах погрешности; порог 1.9.

Помеченный результат не присваивает грейд: кандидату сразу доступна **одна подтверждающая форма** из непоказанных ему заданий (без кулдауна). Честные кандидаты помечаются в 4,4% случаев и просто проходят ещё одну форму.

5. Правила грейдов и кулдауны (`services/testing/policy.py`)

- одна активная попытка; лимит времени 30 минут (просроченные завершаются автоматически с данными ответами);
- тот же (специализация, грейд) — повтор через 30 дней;
- без подтверждённого грейда: провалили Middle → тест на Junior доступен сразу (профиль не блокируется и не удаляется);
- подтверждённый грейд меняется не чаще раза в 90 дней, кроме «быстрой дорожки»: уверенный результат открывает тест на следующий грейд сразу;
- проваленный тест на повышение **не понижает** грейд; переподтверждение текущего грейда обновляет актуальность профиля;
- смена специализации = смена категории (тот же 90-дневный лимит).

Все сроки настраиваются переменными окружения.

6. Устойчивость к распространению заданий — итог

1. Вариант у каждого свой (пересечение соседних вариантов ~30–50% по статическим заданиям), порядок и варианты ответов перемешаны.
2. Параметрические семейства не утекают по построению; их доля в варианте повышена.
3. Контроль экспозиции распределяет показы по банку; исключаются уже виденные задания.
4. IRT позволяет постоянно пополнять банк: новые задания калибруются по ответам (см. п. 7 валидации) и сразу становятся взаимозаменяемыми со старыми.
5. Детектор утечки + подтверждающая форма.
6. Регулярные короткие задания от работодателей дают независимый «свежий» сигнал.

7. Валидация (`python evaluation/evaluate_tests.py`)

Эталонной разметки нет, поэтому валидность проверяется **симуляцией с известной истиной** — стандартная психометрическая процедура:

- у синтетического кандидата есть истинная способность θ, а значит и истинный грейд (400 кандидатов × 4 специализации; 10% ниже Junior; 20% заявляют грейд выше реального);
- ответы генерируются по 3PL с **истинными** параметрами заданий, которые отличаются от номинальных (`b + N(0; 0.35)`, `a · LogN(0; 0.2)`), а движок оценивает по номинальным — проверка не круговая и включает ошибку модели;

- используется продакшн-код целиком: blueprint → генерация → оценка → правила грейдов → подтверждение.

Результаты (seed = 7, длина теста 20):

Метрика	Значение
Точность итогового грейда (= истинному)	**80,5%**
Взвешенная к Коэна	**0,898**
Грейд в пределах ±1	99,9%
Завышение грейда (риск для работодателя)	3,5%
Занижение (кандидат может пересдать / пройти ниже)	16,0%
Тест–ретест: корреляция θ на двух независимых формах	**0,866** (20 заданий)
Тест–ретест: согласие «сдал / не сдал»	79,9%
Эквивалентность форм: SD ожидаемого балла у порога, blueprint / случайный отбор	**0,013 / 0,045**
Дискриминативность: AUC Middle vs Junior · Senior vs Junior	0,957 · 1,000
d Коэна Middle vs Junior	2,32
Доля заданий с корреляцией «задание–остаток» > 0,2	75,4%
Средний балл по истинному грейду: ниже Junior / Junior / Middle / Senior	15 / 36 / 62 / 88

Утечка: кандидат уровня Junior (θ = −0,4) заранее знает ответы K «слитых» вариантов теста Middle. Доля сдавших:

K	Единый тест для всех	Генерируемые формы	Формы + детектор + подтверждение
0	7,7%	8,3%	6,7%
1	98,7%	44,3%	**23,0%**
3	98,3%	82,7%	**12,0%**
10	98,3%	97,0%	**1,3%**

Надёжность по длине теста (корреляция θ на параллельных формах): 10 заданий — 0,72; 15 — 0,80; **20 — 0,87**; 25 — 0,87. Поэтому выбрана длина 20 (≈ 10–15 минут).

Калибровка: по 1 500 симулированным прохождениям трудности заданий переоцениваются по ответам; RMSE относительно истинных b падает с 0,30 (экспертные уровни) до 0,18 — банк самоулучшается по мере накопления данных.

Ограничения: симуляция предполагает одномерную способность внутри специализации; реальная калибровка требует пилота на живых кандидатах (процедура калибровки уже реализована в скрипте). Полный отчёт: `evaluation/results/tests_report.md`.

5. Интеграция с реестром ФСП

API реестра ФСП на хакатоне не предоставляется, поэтому реализованы **порт** и два **адаптера**: платформа зависит только от интерфейса, замена мока на реальную интеграцию не затрагивает бизнес-логику.

```
class FSPRegistry(Protocol):              # backend/app/services/fsp/registry.py
    source: str
    def get_participant(self, fsp_id: str) -> RegistryParticipant | None: ...
    def find_by_email(self, email: str) -> RegistryParticipant | None: ...
```

Адаптер	Режим (`FSP_REGISTRY_MODE`)	Описание
`MockFSPRegistry`	`mock` (по умолчанию)	Синтетический снимок реестра `mock_data/fsp_registry.json`
`HttpFSPRegistry`	`http`	REST-клиент; контракт ниже. Мок-API поднят самим backend: `/api/mock-fsp/v1`

Контракт REST API реестра

```
GET /participants/{fsp_id}          → 200 Participant | 404
GET /participants?email=<e-mail>    → 200 Participant | 404
```

```
{
  "fsp_id": "FSP-100777",
  "full_name": "Демидов Алексей",
  "email": "candidate@demo.ru",
  "region": "г. Москва",
  "sport_rank": "I разряд",
  "achievements": [{
    "id": "ACH-2025-31415",
    "event_name": "Всероссийские соревнования по продуктовому программированию",
    "discipline": "Продуктовое программирование",
    "level": "all_russian",            // regional | interregional | all_russian | international
    "date": "2026-05-08",
    "place": 2, "participants_total": 180,
    "team_name": "Byte Busters", "role": "backend",
    "score": 91.5, "specialization_hint": "backend",
    "certificate_url": "https://registry.fsp.example/certificates/ACH-2025-31415"
  }]
}
```

Набор полей выбран так, чтобы работодатель понимал **уровень** (тип соревнования, место среди скольких участников), **релевантность** (дисциплина, роль в команде) и **проверяемость** (ссылка на сертификат). Дисциплины соответствуют направлениям ФСП: продуктовое, алгоритмическое программирование, информационная безопасность, программирование БАС и робототехники.

Связь профиля с ФСП ID

1. **Автоматически**: после подтверждения e-mail кандидата выполняется поиск в реестре по e-mail (`fsp_service.try_auto_link`). Найден — профиль обогащается без действий пользователя.
2. **Вручную**: кандидат вводит ФСП ID. Если e-mail в реестре совпадает — привязка мгновенная; иначе код подтверждения отправляется на e-mail из реестра (владение подтверждается, чужие достижения присвоить нельзя). Один ФСП ID — один профиль (UNIQUE + проверка, 409).

3. **Целевое состояние**: вход «через ФСП ID» на Keycloak. Токен Keycloak содержит `sub` участника — он сохраняется в `users.external_subject`, шаг с кодом заменяется OIDC-авторизацией.

Данные кешируются локально (`fsp_profiles`, `fsp_achievements`) с отметкой `synced_at` и источником; кнопка «Синхронизировать» перечитывает реестр. Отвязка удаляет локальный снимок.

Как ФСП влияет на ранжирование

Сила сигнала для специализации `s` (`services/fsp/scoring.py`):

```
points(a) = level_weight(a.level) × place_factor(a.place, a.participants_total)
          × recency(0.5 ^ (years / 2.5)) × relevance(a.discipline, a.role, s)
strength  = 1 − exp(−(Σ points + rank_bonus) / 1.2)          ∈ [0, 1)
```

- `level_weight`: международные 1.0, всероссийские 0.85, межрегиональные 0.6, региональные 0.45;
- `place_factor`: 1 место 1.0, 2 — 0.85, 3 — 0.75, топ-10% — 0.55, топ-30% — 0.4, участие — 0.25;
- `relevance`: роль в команде совпадает со специализацией — 1.0, иначе матрица «дисциплина × специализация» (например, ИБ → DevOps 0.9, алгоритмическое → Backend 0.8);
- `rank_bonus`: КМС/МС/разряды.

В Matching Engine компонент ФСП = `0.3 + 0.7 × strength`. **Отсутствие истории ФСП ≠ 0**: используется нейтральная база 0.3, сигнал считается «недоступным», и кандидат не штрафуется; любое участие может только поднять профиль. Валидация: идеальные кандидаты без ФСП в среднем находятся на 4,5-м перцентиле выдачи против 3,6-го у кандидатов с ФСП (см. matching.md). Кандидат может скрыть ФСП в настройках — тогда сигнал не используется и в ранжировании.

UI

- Кандидат: страница «Достижения ФСП» — список с медалями, уровнем, местом, командой, ролью; сила сигнала; привязка/синхронизация/отвязка; явное сообщение «История ФСП не найдена — это нормально».
- Работодатель: бейдж «N достижений ФСП» в карточке, лучшая позиция в объяснении, полный список в профиле; фильтр «есть / нет достижений ФСП» в подборке и банке кандидатов.

6. API

Полная интерактивная спецификация OpenAPI 3.1 (Swagger UI): **http://localhost:8000/api/docs** (ReDoc: `/api/redoc`, JSON: `/api/openapi.json`). Все запросы, ответы и коды ошибок описаны в спецификации; TypeScript-типы фронтенда генерируются из неё (`npm run gen:api`).

- Аутентификация: `Authorization: Bearer <JWT>` — токен из `/auth/login` или `/auth/verify-email`.
- Формат ошибок: `{"code": "cooldown", "message": "…", "details": {...}}`.
- Язык: заголовок `Accept-Language` (`ru` — по умолчанию, `en`). На нём возвращаются сообщения об ошибках (включая 422 с названием поля), объяснения подбора, рекомендации по тесту, справочники и тексты заданий; ответ содержит `Content-Language` и `Vary: Accept-Language`. `code` ошибки от языка не зависит.
- Коды: `401` нет/просрочен токен, `403` чужая роль, `404` не найдено или нет доступа, `409` конфликт состояния (кулдаун, дубликат, закрытое приглашение), `422` валидация, `429` лимит приглашений.

Эндпоинты

Аутентификация

Метод	Путь	Назначение
POST	`/api/auth/register`	Регистрация (роль, согласие 152-ФЗ) → код на e-mail
POST	`/api/auth/verify-email`	Подтверждение кода → JWT
POST	`/api/auth/resend-code`	Повторная отправка кода
POST	`/api/auth/login`	Вход → JWT
GET	`/api/auth/me`	Текущий пользователь (включая `locale` — язык интерфейса)
PUT	`/api/auth/me/locale`	Сохранить язык интерфейса в учётной записи (`ru` / `en`)

Справочники

GET	`/api/reference`	Специализации и компетенции, грейды с порогами, навыки, профили весов

Кабинет кандидата

Метод	Путь	Назначение
GET	`/api/candidate/dashboard`	Категория, заполненность, шаги, приглашения, последний тест
GET/PUT	`/api/candidate/profile`	Профиль, резюме, опрос (специализация, грейд), навыки
POST	`/api/candidate/resume/parse`	Распознавание PDF-резюме (без сохранения)
GET/PUT	`/api/candidate/settings`	Приватность и согласие на публикацию
GET	`/api/candidate/fsp`	Достижения ФСП и сила сигнала
POST	`/api/candidate/fsp/link` · `/fsp/confirm` · `/fsp/sync`	Привязка / подтверждение / синхронизация ФСП ID
DELETE	`/api/candidate/fsp/link`	Отвязка
GET	`/api/candidate/invitations`	Входящие приглашения

Метод	Путь	Назначение
GET	`/api/candidate/invitations/{id}`	Открыть (статус → `viewed`)
POST	`/api/candidate/invitations/{id}/accept` · `/reject` · `/report`	Ответ / жалоба
GET	`/api/candidate/tasks` · POST `/tasks/{id}/submit`	Короткие задания

Тестирование

Метод	Путь	Назначение
GET	`/api/candidate/testing?specialization_id=`	Доступность по грейдам (кулдауны), активная попытка, история, правила
POST	`/api/candidate/testing/start`	Сгенерировать индивидуальный вариант `{specialization_id, target_grade}`
GET	`/api/candidate/testing/attempts/{id}`	Вопросы попытки (без правильных ответов)
POST	`/api/candidate/testing/answer`	Сохранить ответ `{attempt_id, item_id, answer: [...]}`
POST	`/api/candidate/testing/finish`	Завершить → грейд, категория, компетенции, уверенность
GET	`/api/candidate/testing/results/{attempt_id}`	Результат

Кабинет работодателя

Метод	Путь	Назначение
GET	`/api/employer/dashboard`	Потребности, воронка приглашений, последние ответы
GET/PUT	`/api/employer/profile`	Профиль компании
GET/POST	`/api/employer/requirements`	Потребности; создание сразу формирует подборку
POST	`/api/employer/requirements/parse`	Текст потребности → специализация, грейд, стек, вилка
GET/PUT/DELETE	`/api/employer/requirements/{id}`	Потребность: просмотр / уточнение (пересчёт) / закрытие
GET	`/api/employer/requirements/{id}/matches`	Подборка: `grades, categories, skills, has_fsp, min_score, shortlisted, refresh`
POST	`/api/employer/requirements/{id}/shortlist/{candidate_id}?value=`	Избранное
GET	`/api/employer/requirements/{id}/compare?a=&b=`	Почему A выше B
GET	`/api/employer/candidates`	Банк кандидатов с фильтрами
GET	`/api/employer/candidates/{id}?requirement_id=`	Профиль + объяснение; контакты — по правилам видимости

Метод	Путь	Назначение
GET/POST	`/api/employer/invitations`	История / отправка приглашения (зарплата обязательна)
POST	`/api/employer/invitations/{id}/withdraw`	Отозвать
GET	`/api/employer/applications` · POST `/applications/{id}/status`	Отклики на вакансии
GET/POST	`/api/employer/tasks` · GET `/tasks/{id}/submissions` · POST `/submissions/{id}/review`	Короткие задания

Вакансии и отклики

GET	`/api/jobs`	Опубликованные вакансии (для кандидата — с % совпадения, обратный матчинг)
GET	`/api/jobs/{id}`	Вакансия
POST	`/api/applications`	Отклик кандидата (раскрывает контакты этому работодателю)
GET	`/api/applications`	Мои отклики

Мок реестра ФСП

GET	`/api/mock-fsp/v1/participants/{fsp_id}`	Участник
GET	`/api/mock-fsp/v1/participants?email=`	Поиск по e-mail
GET	`/api/mock-fsp/v1/stats`	Размер мок-реестра

Пример: потребность → подборка → приглашение

```
TOKEN=$(curl -s -X POST localhost:8000/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"hr@demo.ru","password":"demo12345"}' | jq -r .access_token)

curl -s -X POST localhost:8000/api/employer/requirements -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"title":"Backend Developer","specialization_id":"backend",
  "grade_id":"middle","must_skills":["python","fastapi","postgresql","docker"],
  "salary_from":180000,"salary_to":250000}' | jq .id

curl -s "localhost:8000/api/employer/requirements/13/matches?limit=3" -H "Authorization: Bearer $TOKEN" \
  | jq '.items[] | {name: .display_name, category: .category.label, percent: .match.percent,
        why: [.match.reasons[].text]}'
```

Фрагмент ответа:

```
{
  "name": "Алексей Д.", "category": "Backend · Middle", "percent": 93,
  "why": ["Специализация Backend совпадает", "Подтверждённый грейд Middle — как требуется",
          "Совместимость стека 90%: 4 из 4 обязательных навыков, 3 подтверждены тестом",
          "Тест: 76/100 (уверенность 100%)",
          "3 подтверждённых достижения ФСП: 2 место — Всероссийские соревнования по продуктовому
программированию (2026)"]
}
```

7. Сборка и развёртывание

Вариант 1. Docker Compose (рекомендуется)

Требования: Docker 24+ с Compose v2, ~1 ГБ свободной памяти.

```
cp .env.example .env        # необязательно: значения по умолчанию работают
docker compose up --build
```

Сервис	Адрес	Что это
frontend	http://localhost:8080	Веб-приложение (nginx, проксирует `/api` в backend)
backend	http://localhost:8000/api/docs	API + Swagger UI
mailpit	http://localhost:8025	Все письма платформы (коды подтверждения)
db	внутренняя сеть	PostgreSQL 16, том `pgdata`

При старте backend ждёт БД, применяет миграции Alembic и **один раз** заполняет пустую базу демо-данными (`python -m app.seed --if-empty`). Повторный запуск данные не трогает.

Полезные команды:

```
docker compose logs -f backend                                  # логи API
docker compose exec backend python -m app.seed --reset          # пересоздать демо-данные
docker compose exec backend pytest -q                           # тесты внутри контейнера
docker compose down -v                                          # остановить и удалить данные
python scripts/smoke_test.py --base http://localhost:8080       # сквозной smoke-тест по HTTP
```

Вариант 2. Локально без Docker

Требования: Python 3.12, Node.js 20+. По умолчанию используется SQLite (`backend/fsp_talent.db`), PostgreSQL не нужен.

```
# Backend
cd backend
python -m venv .venv
.venv/Scripts/activate            # Windows;  source .venv/bin/activate — Linux/macOS
pip install -r requirements-dev.txt
alembic upgrade head
python -m app.seed                # демо-данные (можно с --reset)
uvicorn app.main:app --port 8000 --reload

# Frontend (в другом терминале)
cd frontend
npm ci
npm run dev                        # http://127.0.0.1:5173, /api проксируется на :8000
```

Чтобы работать с PostgreSQL локально, задайте
`DATABASE_URL=postgresql+psycopg://fsp:fsp_password@localhost:5432/fsp_talent` .

Проверки качества

```
cd backend && pytest -q && ruff check app tests scripts      # 381 тест, линтер
cd frontend && npm test                 # 107 тестов: i18n, API-клиент, переключение языка, страницы
cd frontend && npm run typecheck && npm run lint && npm run build
python evaluation/evaluate_tests.py      # валидация механики тестирования (~2 мин)
python evaluation/evaluate_matching.py   # бенчмарк подбора (~30 с)
python evaluation/tune_matching.py       # подбор гиперпараметров (~4 мин)
```

Скрипты оценки не требуют БД и сервера — используют те же модули движков, что и API. Результаты сохраняются в `evaluation/results/*.md|json`.

Переменные окружения

Полный список с комментариями — в `.env.example`. Основные:

Переменная	По умолчанию	Назначение
`DATABASE_URL`	SQLite в `backend/`	Строка подключения SQLAlchemy
`JWT_SECRET`	dev-значение	**Обязательно сменить в продакшене**
`EMAIL_DEV_EXPOSE_CODE`	`true`	Демо-режим: код подтверждения виден в UI. В продакшене — `false`
`SMTP_HOST` / `SMTP_PORT`	— / 1025	SMTP (в compose — Mailpit)
`TEST_LENGTH`, `TEST_TIME_LIMIT_MINUTES`	20, 30	Параметры теста
`RETAKE_COOLDOWN_DAYS`, `GRADE_CHANGE_COOLDOWN_DAYS`	30, 90	Кулдауны
`FSP_REGISTRY_MODE`, `FSP_REGISTRY_URL`	`mock`	Источник данных ФСП: `mock` или `http`
`MATCHING_WEIGHTS_JSON`	—	Переопределение весов матчинга
`INVITATIONS_DAILY_LIMIT`	40	Анти-спам лимит приглашений

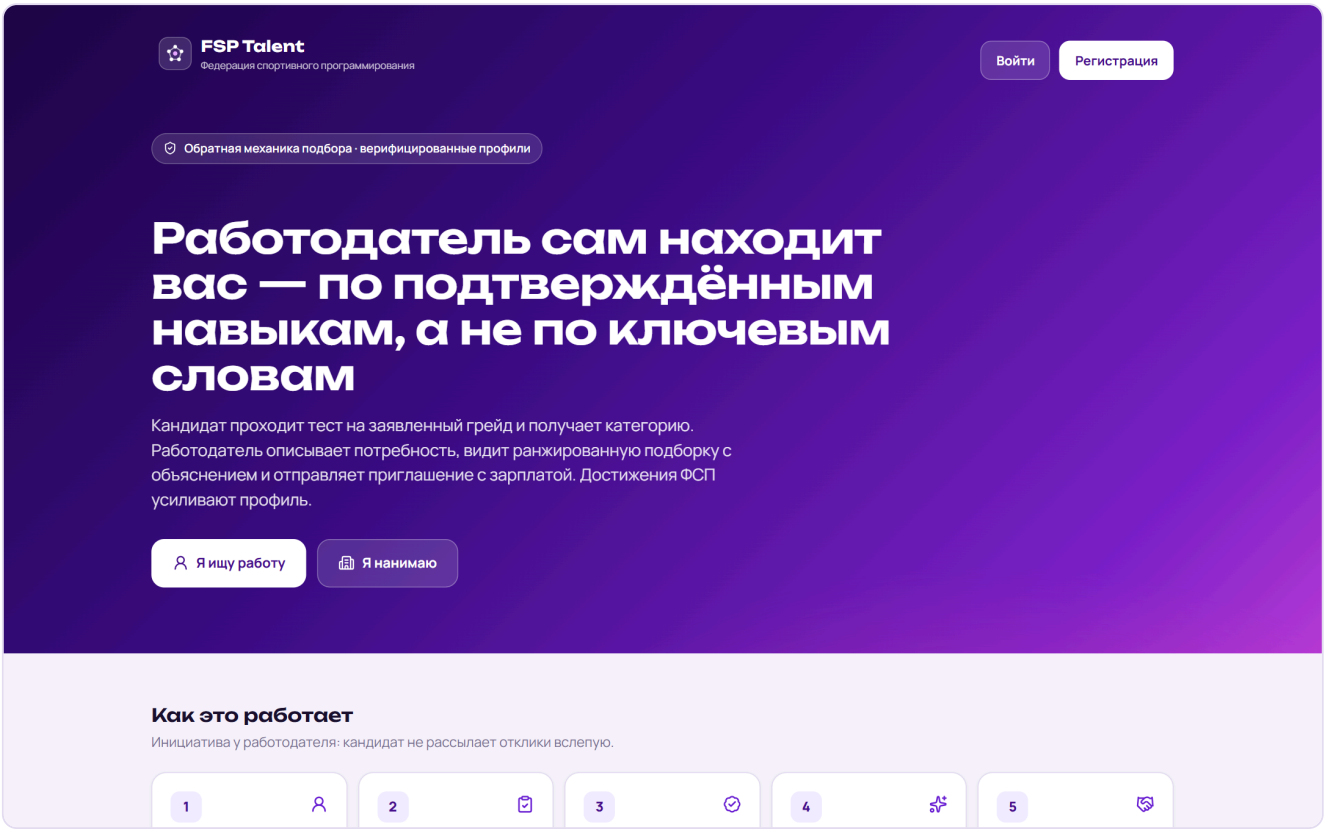
Развёртывание на сервере

Достаточно одной ВМ (2 vCPU / 2 ГБ): `docker compose up -d --build`, затем поставить перед портом 8080 TLS-терминатор (nginx/Caddy/Traefik) и закрыть внешний доступ к 8000/8025. Перед публичным запуском: `JWT_SECRET`, `POSTGRES_PASSWORD`, `EMAIL_DEV_EXPOSE_CODE=false`, реальный SMTP.

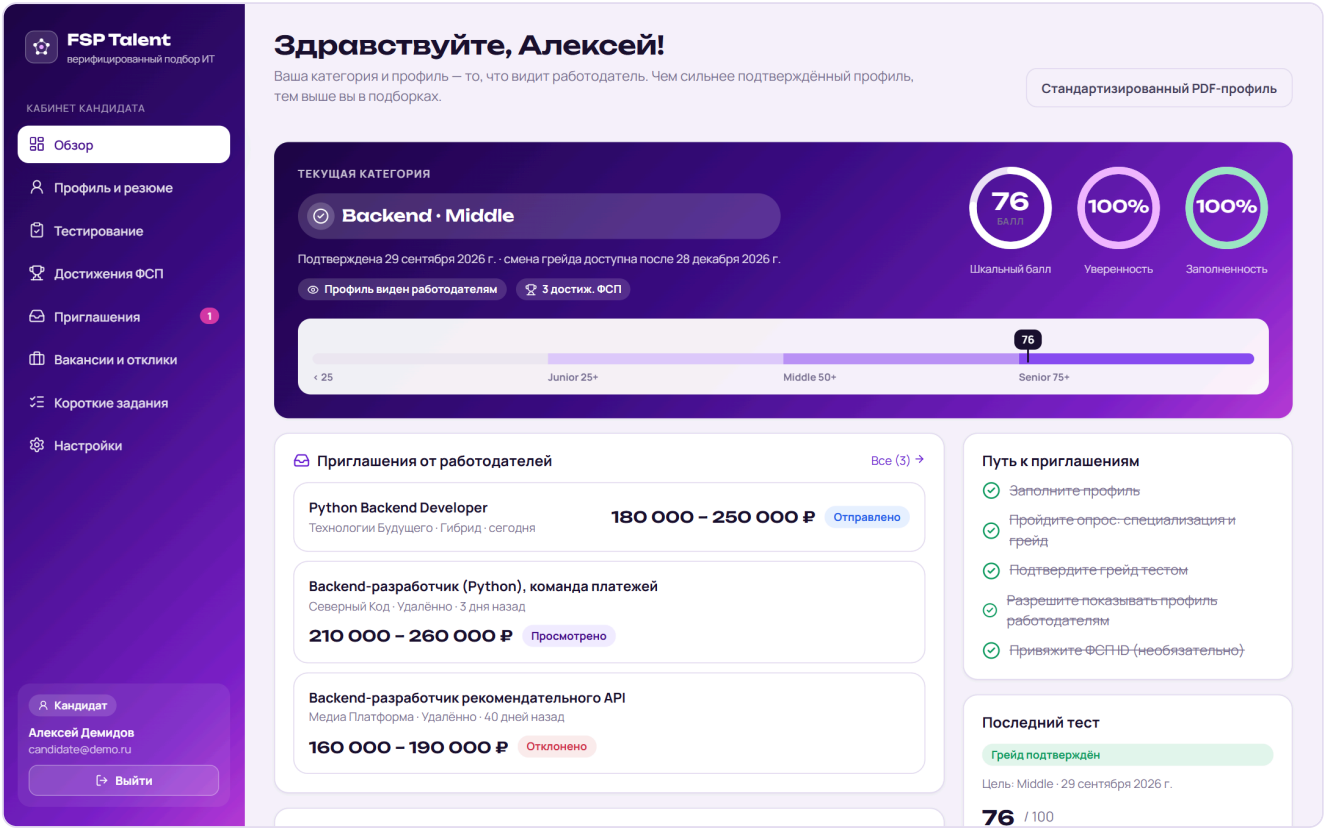
Типовые проблемы

- **Порт занят** (8000/8080/8025) — измените публикацию портов в `docker-compose.yml`.
- **Vite не открывается по `localhost`** на Windows — используйте `http://127.0.0.1:5173` (dev-сервер слушает IPv4).
- **Пустая выдача у работодателя** — кандидат попадает в подборки только после подтверждённой категории и согласия на публикацию профиля (Настройки → «Публикация профиля»).

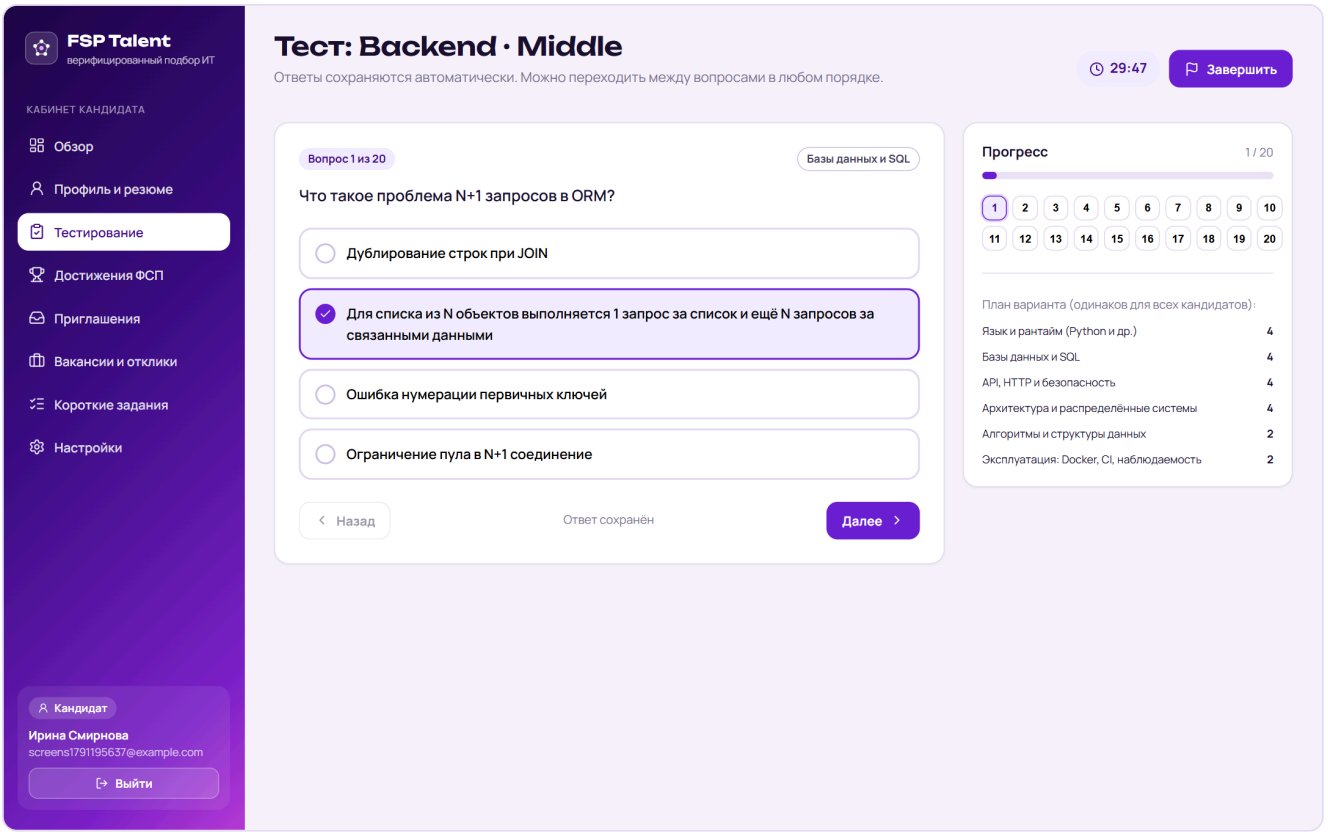
8. Интерфейс



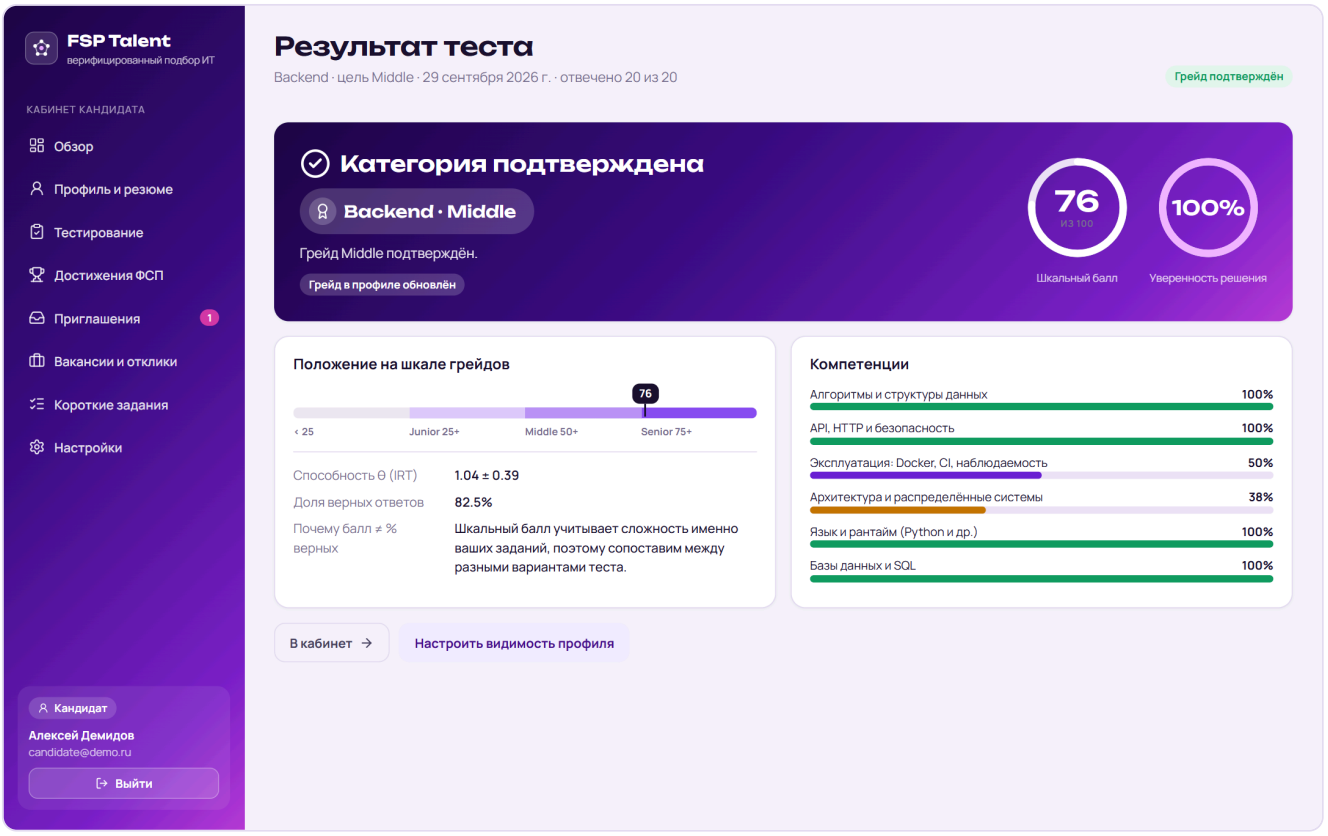
Главная страница: обратная механика подбора и демо-доступ



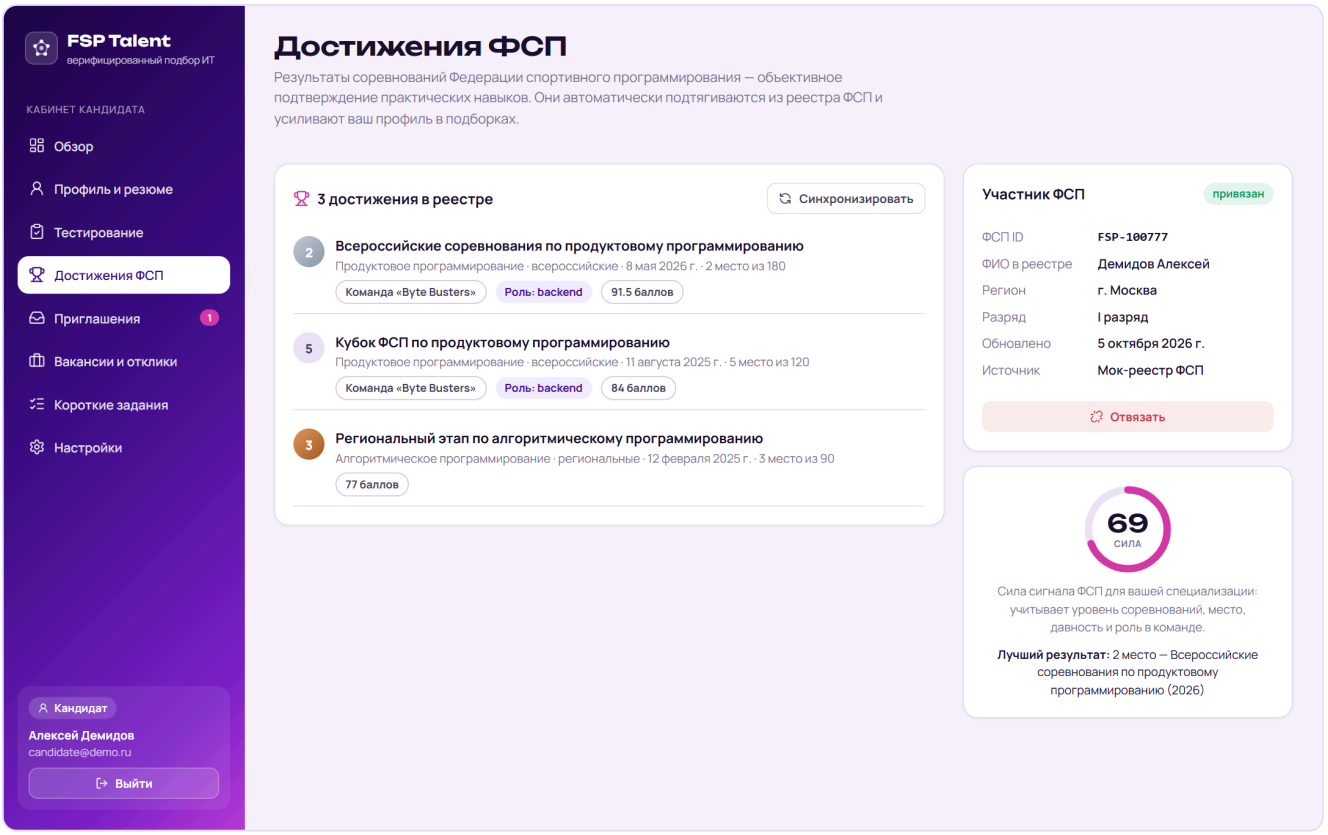
Кабинет кандидата: подтверждённая категория, шкальный балл, приглашения



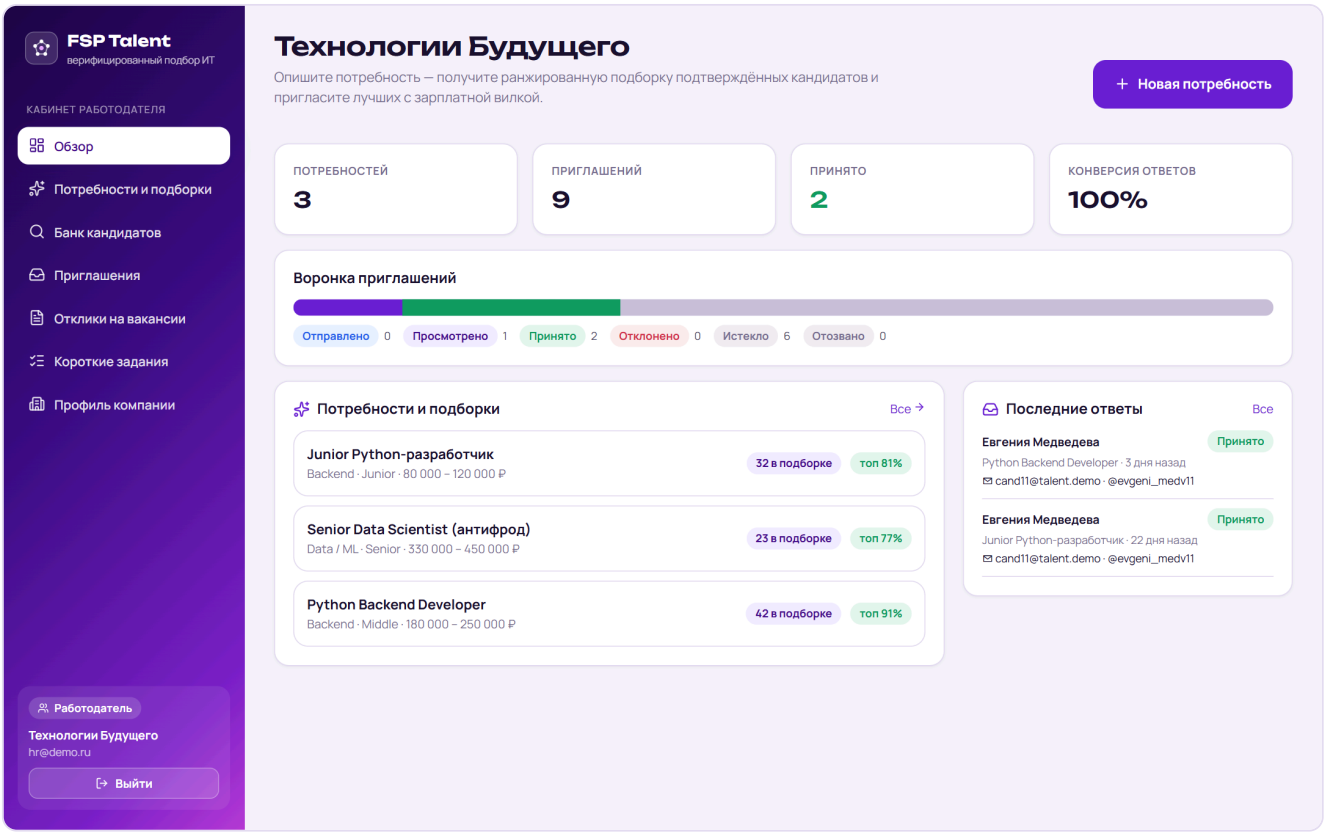
Прохождение теста: индивидуальный вариант по единому плану, таймер, автосохранение



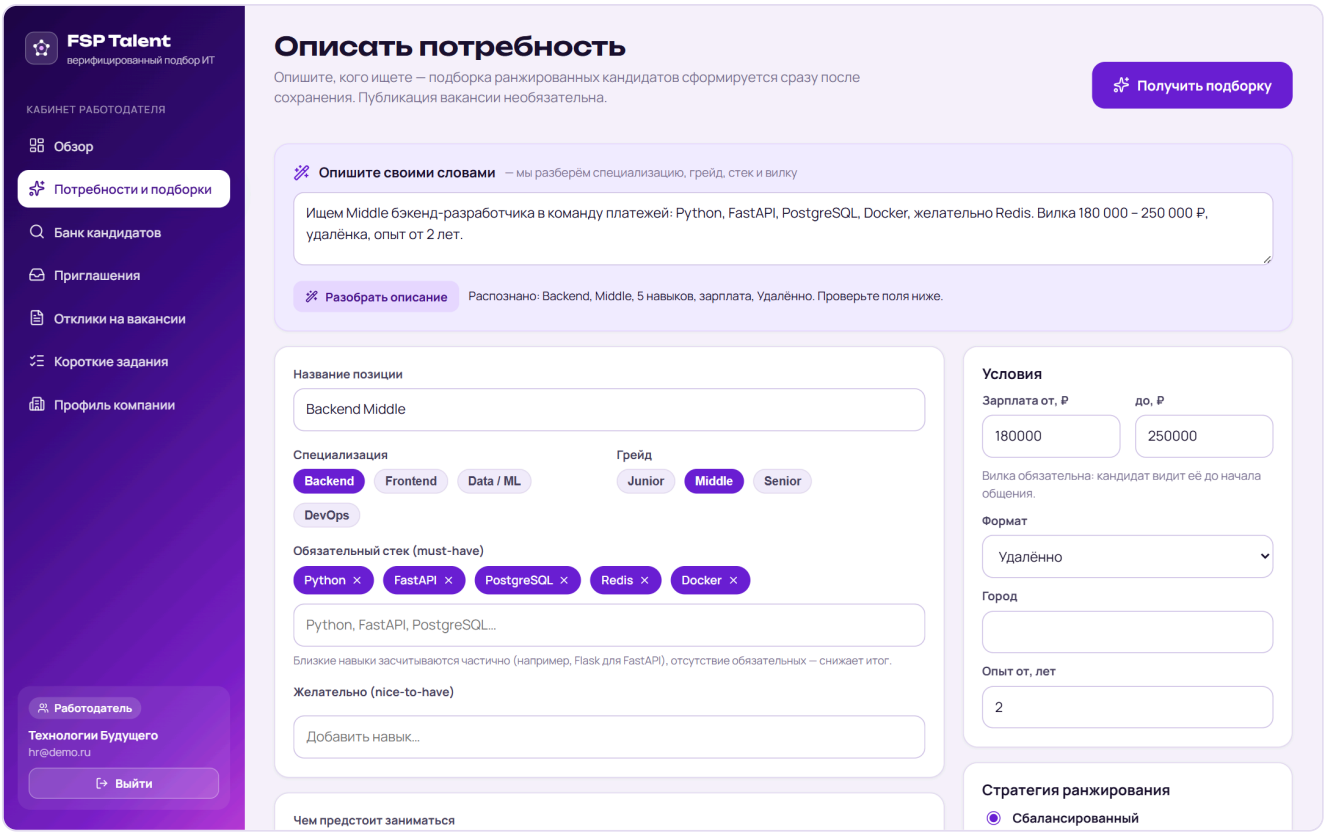
Результат теста: категория, положение на шкале грейдов, компетенции, уверенность



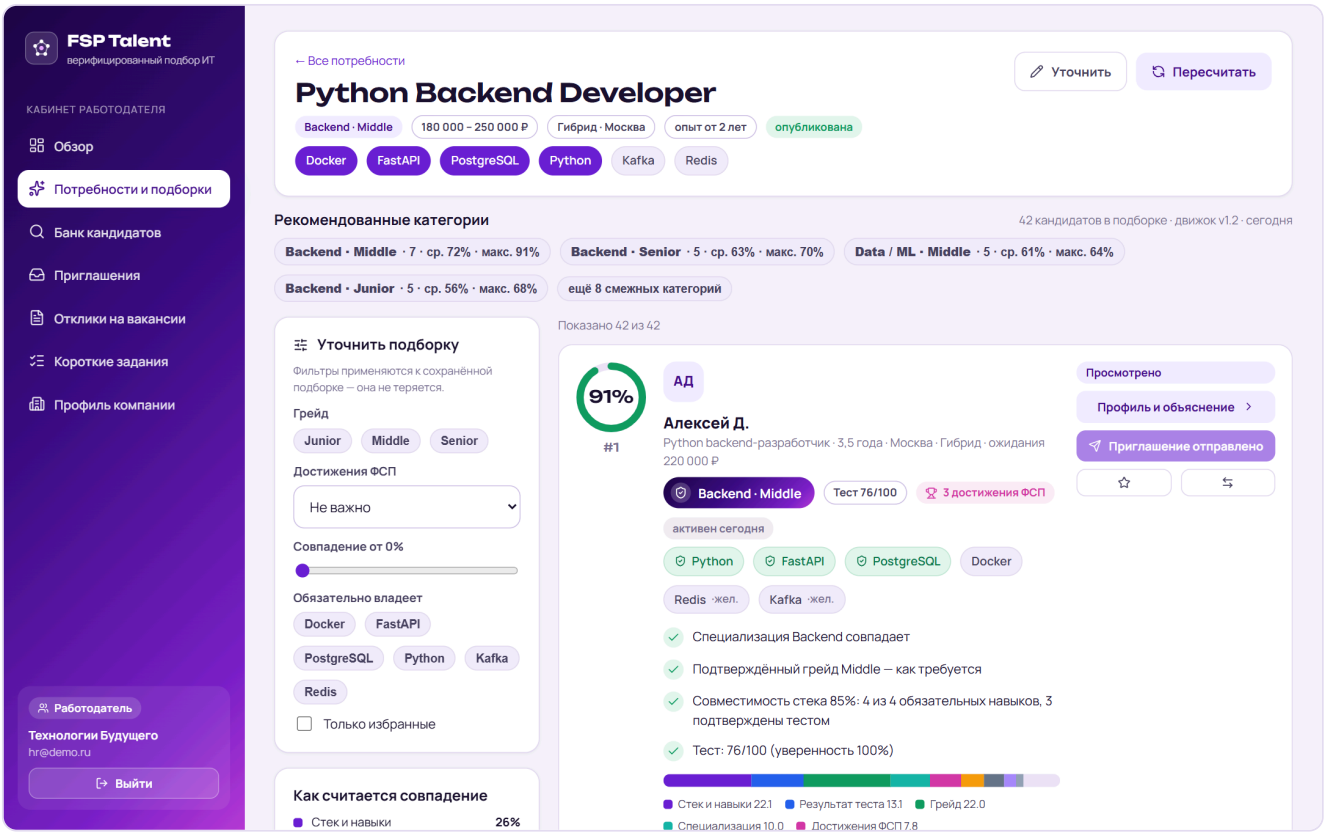
Достижения ФСП: автоматическое обогащение профиля из реестра



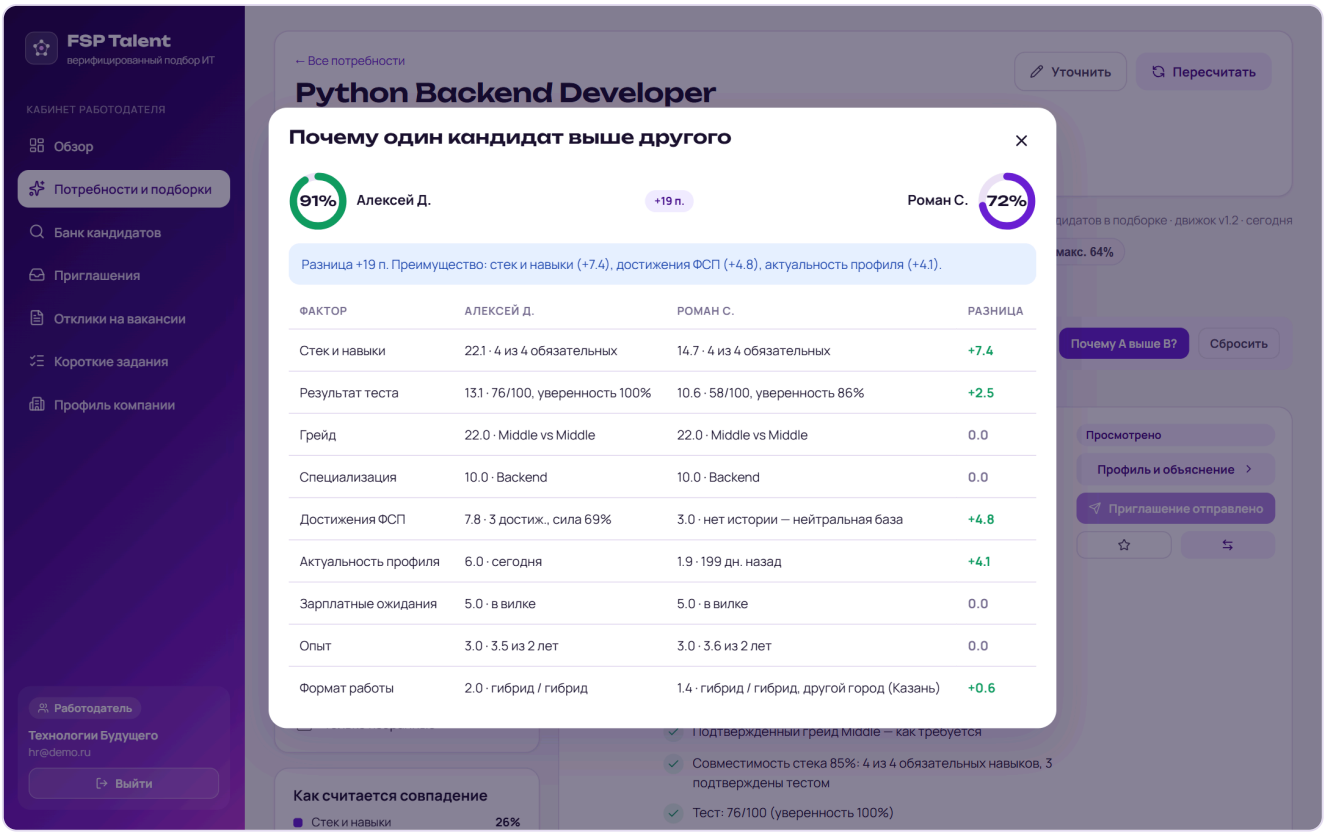
Кабинет работодателя: потребности, воронка приглашений



Описание потребности: разбор текста в специализацию, грейд, стек и вилку



Подборка: рекомендованные категории, рейтинг, объяснение, вклад факторов



Почему кандидат А выше кандидата В

FSP Talent
верифицированный подбор ИТ

КАБИНЕТ РАБОТОДАТЕЛЯ

- Обзор
- Потребности и подборки
- Банк кандидатов
- Приглашения
- Отклики на вакансии
- Короткие задания
- Профиль компании

Работодатель
Технологии Будущего
hr@demo.ru
Выйти

← К подборке «Python Backend Developer»

АД

Алексей Д.

Python backend-разработчик · 3,5 года опыта · Москва · Гибрид

Backend · Middle

3 достижения ФСП · 2 призовых

91%

совпадение с потребностью

Приглашение отправлено

Ожидания: 220 000 ₽

Почему этот кандидат в подборке

для «Python Backend Developer»

- Специализация Backend совпадает
- Подтверждённый грейд Middle — как требуется
- Совместимость стека 85%: 4 из 4 обязательных навыков, 3 подтверждены тестом
- Тест: 76/100 (уверенность 100%)
- 3 подтверждённых достижения ФСП: 2 место — Всероссийские соревнования по продуктовому программированию (2026)
- Профиль активен сегодня
- Ожидания 220 000 ₽ — в вилке

Стек: обязательные и желательные навыки

Python · FastAPI · PostgreSQL · Docker · Redis ·жел. · Kafka ·жел.

Зелёный — подтверждён тестом · жёлтый — близкий навык засчитан частично · красный — отсутствует

Вклад факторов в 91%

Стек и навыки 22.1 · Результат теста 13.1 · Грейд 22.0 · Специализация 10.0 · Достижения ФСП 7.8 · Актуальность профиля 6.0 · Зарплатные ожидания 5.0 · Опыт 3.0 · Формат работы 2.0

ФАКТОР	ОЦЕНКА	ВЕС	ВКЛАД	ДЕТАЛИ
Стек и навыки	85	26%	22.1	4 из 4 обязательных
Результат теста	82	16%	13.1	76/100, уверенность 100%
Грейд	100	22%	22.0	Middle vs Middle
Специализация	100	10%	10.0	Backend
Достижения ФСП	78	10%	7.8	3 достиж., сила 69%
Актуальность профиля	100	6%	6.0	сегодня
Зарплатные ожидания	100	5%	5.0	в вилке
Опыт	100	3%	3.0	3.5 из 2 лет
Формат работы	100	2%	2.0	гибрид / гибрид

Подтверждённый уровень

тест от 29 сентября 2026 г.

76

‹ 25 · Junior 25+ · Middle 50+ · Senior 75+

Компетенции

- Алгоритмы и структуры данных — 100%
- API, HTTP и безопасность — 100%
- Эксплуатация: Docker, CI, наблюдаемость — 50%
- Архитектура и распределённые системы — 38%
- Язык и рантайм (Python и др.) — 100%
- Базы данных и SQL — 100%

История тестирования

Backend · Middle — 76 — Грейд подтверждён

Достижения ФСП

2 **Всероссийские соревнования по продуктовому программированию**
Продуктовое программирование · всероссийские · 8 мая 2026 г. · 2 место из 180
Команда «Byte Busters» · Роль: backend · 91.5 баллов

5 **Кубок ФСП по продуктовому программированию**
Продуктовое программирование · всероссийские · 11 августа 2025 г. · 5 место из 120
Команда «Byte Busters» · Роль: backend · 84 баллов

3 **Региональный этап по алгоритмическому программированию**
Алгоритмическое программирование · региональные · 12 февраля 2025 г. · 3 место из 90
77 баллов

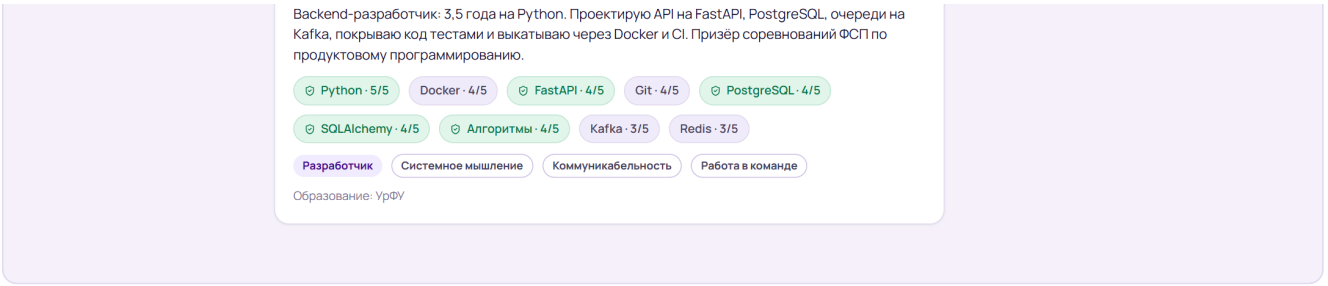
О кандидате

Контакты

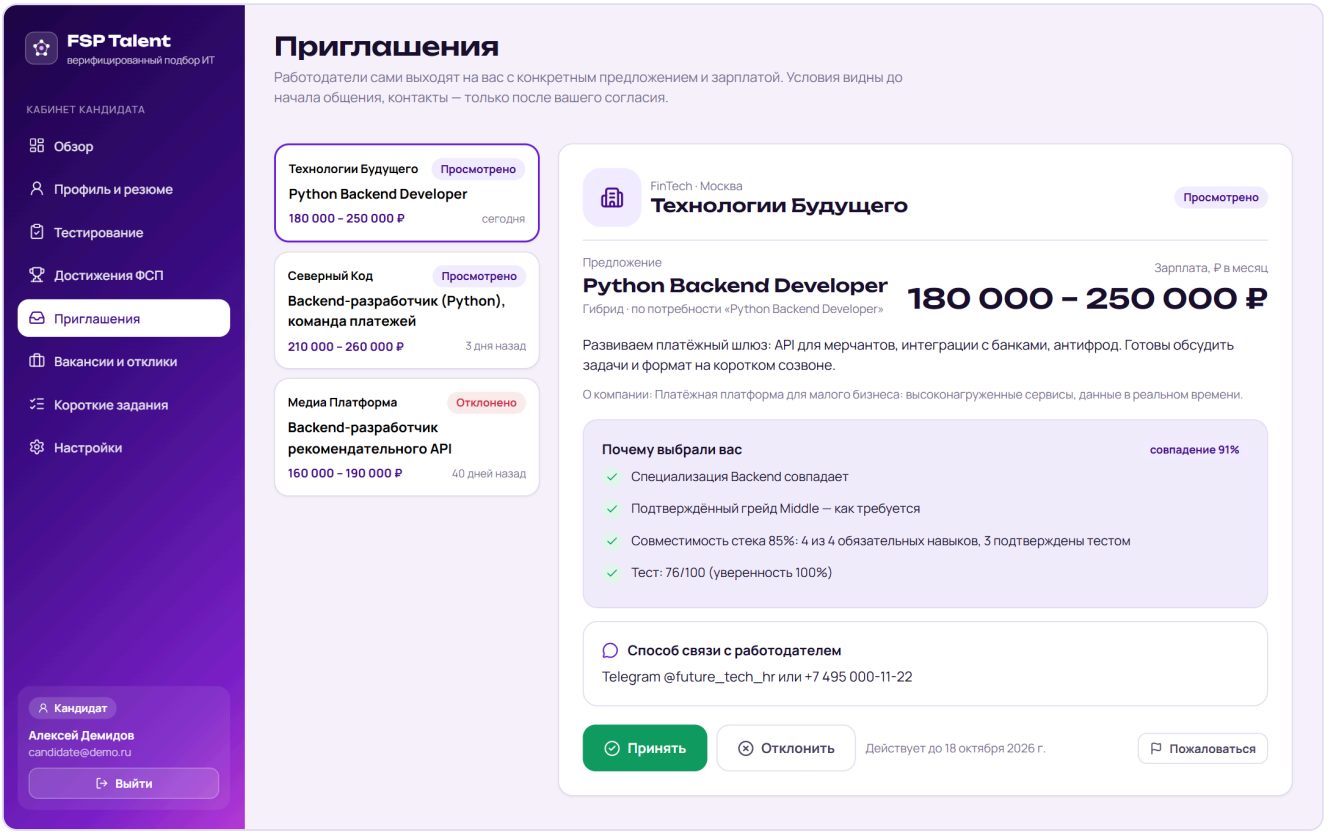
Скрыты до тех пор, пока кандидат не примет ваше приглашение или не откликнется сам. Так кандидат видит ваши условия до начала общения.

Ваши приглашения

Python Backend Developer — Просмотрено
180 000 – 250 000 ₽ · сегодня



Профиль кандидата для работодателя: обьяснение совпадения, контакты скрыты



Приглашение у кандидата: компания, предложение и зарплата до ответа